

Implementacija redkih matrik in metode SOR

Matej Rupnik

May 27, 2026

1 Opis problema

Obravnavamo reševanje linearnega sistema

$$Ax = b,$$

kjer je $A \in \mathbb{R}^{n \times n}$ razpršena matrika. Ker je pri takšnih matrikah večina elementov enaka nič, klasična gosta predstavitev matrike ni učinkovita ne prostorsko ne računsko. Zato shranjujemo le neničelne elemente in njihove indekse.

Za reševanje sistema uporabimo iterativno metodo SOR (Successive Over-Relaxation), ki predstavlja razširitev Gauss–Seidlove metode z relaksacijskim parametrom ω . Pri izbiri

$$\omega = 1$$

dobimo Gauss–Seidlovo metodo, medtem ko vrednosti

$$0 < \omega < 1$$

predstavljajo podrelaksacijo, vrednosti

$$1 < \omega < 2$$

pa nadrelaksacijo, ki lahko pospeši konvergenco metode. Iteracije ustavimo, ko velja

$$\|Ax^{(k)} - b\|_{\infty} < \delta.$$

Metodo nato uporabimo pri fizikalni vložitvi grafov v ravnino oziroma prostor.

2 Redka matrika

Razpršene matrike predstavimo s strukturo tipa `RedkaMatrika`:

```
struct RedkaMatrika
    V::Vector{Vector{Float64}}
    I::Vector{Vector{Int}}
    n::Int
end
```

Vsaka vrstica matrike je predstavljena z dvema seznamoma:

- `V[i]` vsebuje neničelne vrednosti v vrstici i ,
- `I[i]` vsebuje pripadajoče stolpčne indekse.

Takšna predstavitev ustreza formatu LIL (List of Lists), ki je posebej primeren za iterativne metode in množenje matrike z vektorjem.

Implementirali smo osnovne operacije nad matrikami:

- indeksiranje elementov,
- spreminjanje vrednosti,
- množenje z vektorjem,
- seštevanje, odštevanje in množenje s skalarjem,
- pretvorbo med gosto in razpršeno predstavitvijo.

Najpomembnejša operacija za metodo SOR je množenje matrike z vektorjem:

```
function Base.:*(A::RedkaMatrika, x::Vector{Float64})
    y = zeros(A.n)

    for i in 1:A.n
        for (val, j) in zip(A.V[i], A.I[i])
            y[i] += val * x[j]
        end
    end

    return y
end
```

Ker upoštevamo le neničelne elemente, je izračun učinkovitejši kot pri gosti predstavitvi matrike.

3 SOR metoda

Za reševanje sistema

$$Ax = b$$

uporabimo iterativno metodo SOR. Metoda v vsaki iteraciji zaporedoma posodobi komponente vektorja x , pri čemer uporabi trenutno najboljši približek rešitve. Iteracija se ustavi, ko velja

$$\|Ax^{(k)} - b\|_{\infty} < \delta.$$

Implementacija metode:

```
function sor(A::RedkaMatrika,
            b::Vector{Float64},
            omega::Float64=1.0,
            tol=1e-10)

    max_iterations = 10_000
    n, _ = size(A)
    x = zeros(n)

    for iteration in 1:max_iterations
        for row in 1:n
            diagonal_entry, row_sum = rowTerms(A, row, x)

            x[row] = (1 - omega) * x[row] +
                    (omega / diagonal_entry) *
                    (b[row] - row_sum)
        end

        residual = A * x - b

        if norm(residual, Inf) < tol
            return x, iteration
        end
    end

    error("Convergence not achieved")
end
```

Za posamezno vrstico matrike posebej ločimo diagonalni element od ostalih prispevkov. To omogoča pomožna funkcija `rowTerms`, ki rekonstruira prispevek trenutne vrstice pri posodobitvi komponente vektorja x .

4 Fizikalna vložitev grafov

Metodo SOR uporabimo tudi pri fizikalni vložitvi grafov v ravnino. Vozlišča grafa obravnavamo kot točke, povezane z vzmetmi, njihove lege pa določimo iz ravnotežnih pogojev sistema. To vodi do reševanja razpršenega linearnega sistema z Laplaceovo matriko grafa, ki je za večino grafov redka.

4.1 Krožna lestev

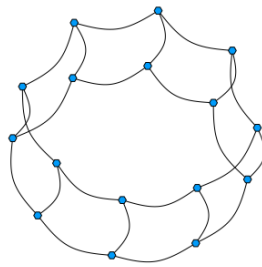


Figure 1: Začetna struktura krožne lestve.

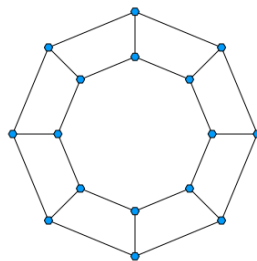


Figure 2: Vložitev krožne lestve po uporabi metode SOR.

4.2 Vložitev 2D mreže

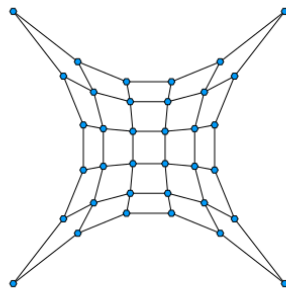


Figure 3: Fizikalna vložitev 2D mreže.

5 Testiranje

Pravilnost implementacije smo preverili z avtomatskimi testi za podatkovni tip `RedkaMatrika` in metodo SOR.

Pri redki matriki smo preverili:

- pretvorbo med gosto in razpršeno predstavitvijo,
- indeksiranje elementov,
- množenje z vektorjem,
- osnovne aritmetične operacije.

Pri metodi SOR smo preverili:

- konvergenco na diagonalno dominantnih sistemih,
- pravilnost glede na referenčno rešitev,
- vpliv parametra ω ,
- ustavitveni pogoj,
- obnašanje pri ničelnem diagonalnem elementu.

6 Rezultati

Implementacija je pravilno rešila vse testne sisteme in dosegla zahtevano toleranco. Posebej smo opazovali vpliv parametra ω na hitrost konvergence. Pokazalo se je, da ustrezna izbira nadrelaksacije lahko bistveno zmanjša število iteracij.

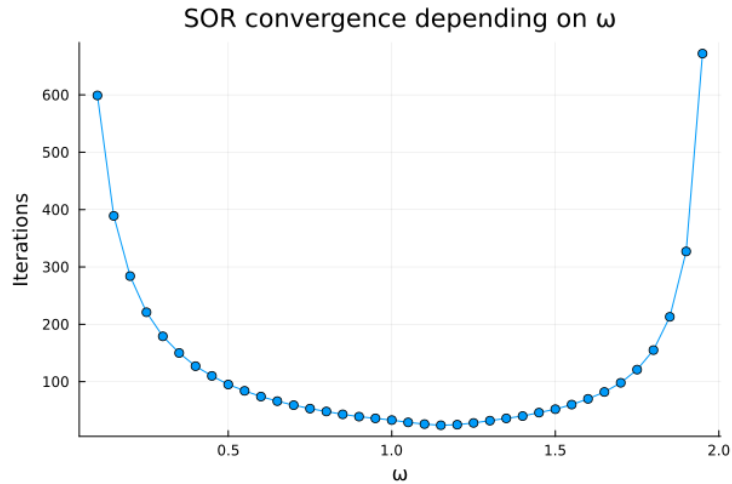


Figure 4: Vpliv parametra ω na število iteracij metode SOR.

V naših primerih se je optimalna vrednost parametra pojavila pri približno

$$\omega \approx 1.1 - 1.2,$$

kjer je metoda konvergirala najhitreje.